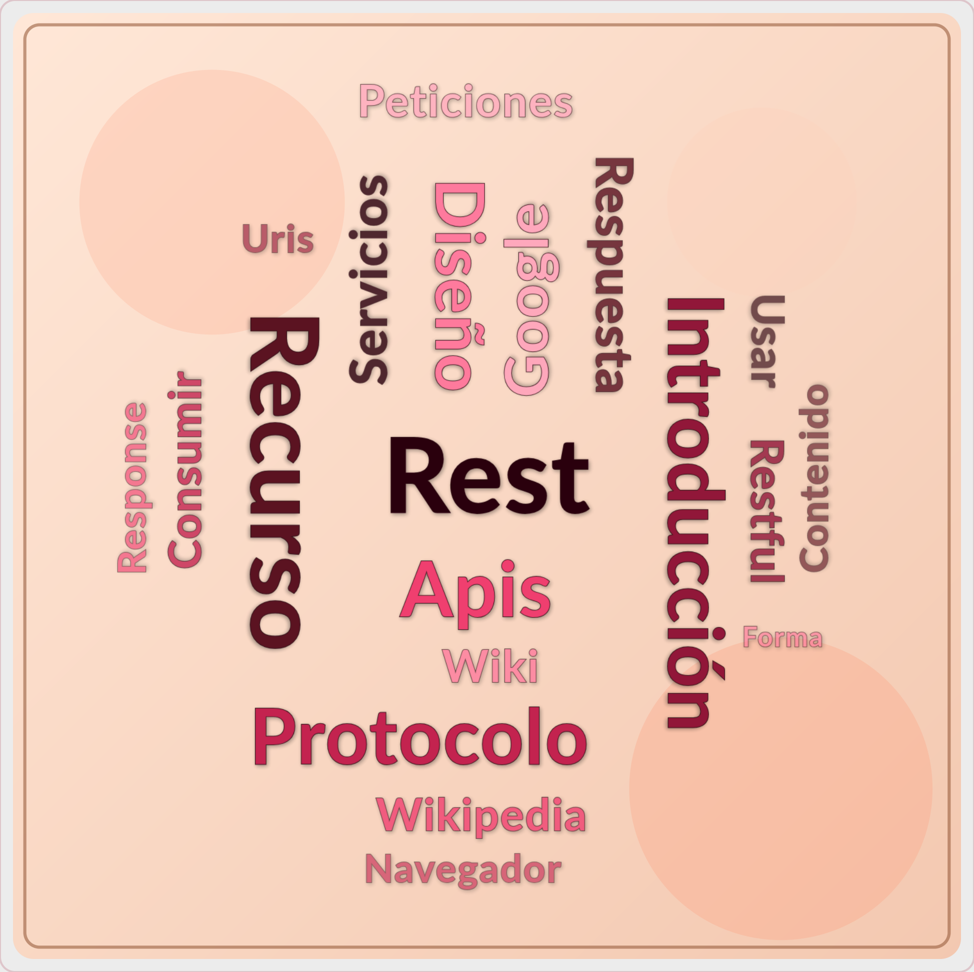
Servicios RESTful

Universidad de Sevilla

Departamento de Lenguajes y Sistemas Informáticos

Carlos Arévalo, Daniel Ayala, José Calderón, Margarita Cruz, Inma Hernández, David Ruiz

Objetivos


Peticiones
Uris
Servicios
Diseño
Google
Respuesta
Introducción
Usar
Recurso
Consumir
Response
Rest
Restful
Contenido
Apis
Forma
Wiki
Protocolo
Wikipedia
Navegador

Contenido

Contenido

Introducción: API

Application Programming Interface

Forma en la que los programas y/o sitios Web intercambian datos (formato JSON o XML)

Sirven para:

- Ofrecer datos y servicios a aplicaciones externas (móvil, Web, consola, …)
- Ofrecer datos y servicios a otros desarrolladores
- Consumir datos de otros servicios (Twitter, Google, Facebook, YouTube, Amazon, Discord, etc.)

https://rapidapi.com/collection/list-of-free-apis

Introducción: API

JSON vs XML

http://localhost:8080/Json/SyncReply/Contacts

```
{
  - Contacts: [
      - {
            FirstName: "Demis",
            LastName: "Bellot",
            Email: "demis.bellot@gmail.com"
        },
      - {
            FirstName: "Steve",
            LastName: "Jobs",
            Email: "steve@apple.com"
        },
      - {
            FirstName: "Steve",
            LastName: "Ballmer",
            Email: "steve@microsoft.com"
        },
      - {
            FirstName: "Eric",
            LastName: "Schmidt",
            Email: "eric@google.com"
        },
      - {
            FirstName: "Larry",
            LastName: "Ellison",
            Email: "larry@oracle.com"
        }
    ]
}
```

http://localhost:8080/Xml/SyncReply/Contacts

```
<ContactsResponse xmlns:i="http://www.w3.org/20
  <Contacts>
    <Contact>
      <Email>demis.bellot@gmail.com</Email>
      <FirstName>Demis</FirstName>
      <LastName>Bellot</LastName>
    </Contact>
    <Contact>
      <Email>steve@apple.com</Email>
      <FirstName>Steve</FirstName>
      <LastName>Jobs</LastName>
    </Contact>
    <Contact>
      <Email>steve@microsoft.com</Email>
      <FirstName>Steve</FirstName>
      <LastName>Ballmer</LastName>
    </Contact>
    <Contact>
      <Email>eric@google.com</Email>
      <FirstName>Eric</FirstName>
      <LastName>Schmidt</LastName>
    </Contact>
    <Contact>
      <Email>larry@oracle.com</Email>
      <FirstName>Larry</FirstName>
      <LastName>Ellison</LastName>
    </Contact>
  </Contacts>
</ContactsResponse>
```

Introducción: REST

La World Wide Web (WWW, W^3, la Web) es un sistema de documentos de hipertexto interconectados accesibles a través de Internet.

En el año 2000, **Roy T. Fielding** estudió en su tesis por qué la Web había sido tan exitosa y enunció una serie de principios generales que generalizaban su arquitectura: **RE**presentational **S**tate **T**ransfer (REST).

REST es un estilo arquitectónico para sistemas distribuidos basados en contenidos hipermedia como la Web y operaciones sin estado.

El principal beneficio de REST es la interoperabilidad entre sistemas en la Web

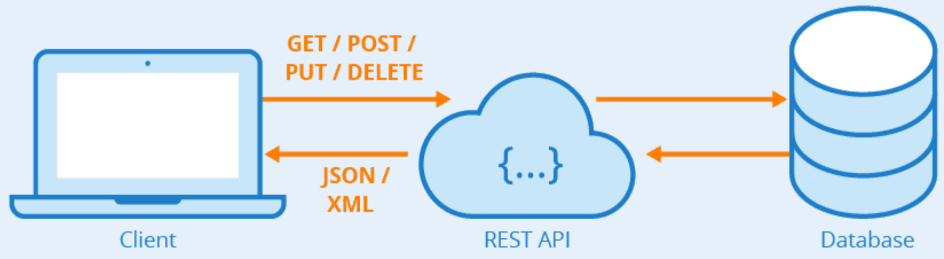
http://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm

https://en.wikipedia.org/wiki/Representational_state_transfer

Introducción: API RESTful

API que sigue los principios REST

- Llamadas a través de peticiones HTTP
- Métodos HTTP según la operación
- Resultado como código de estado HTTP
- Interfaz uniforme
- Peticiones sin estado
- Cacheable
- Separación cliente y servidor



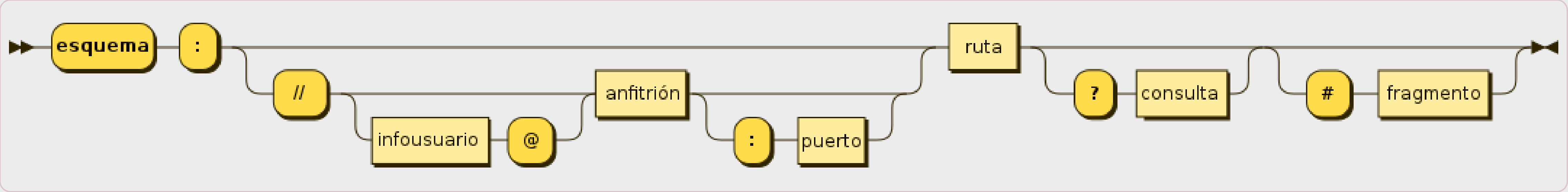
Introducción: Principios REST

Recurso: cualquier cosa que expongamos en la Web, una imagen, un vídeo, un libro, un documento HTML, etc.

URI: (Uniform Resource Identifier) identifica de manera única a un recurso web y, al mismo tiempo, lo hace direccionable y manipulable utilizando un protocolo de aplicación como HTTP

Contenido

Introducción: URI



https://mariadb.com:443/kb/en/library/mysql-command-line-client/#the-mysql_history-file

Esquema: protocolo de acceso (http, https, ftp, ftps, smtp, …)

Host: identificación jerárquica que asigna la autoridad de nombres.

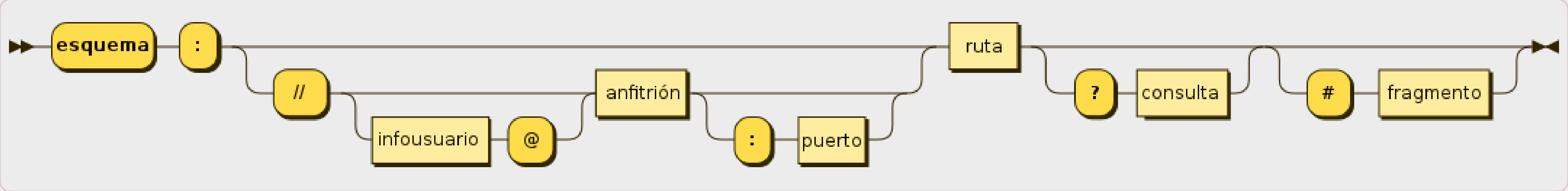
Puerto: número entre 0 y 216-1 (65 535) (puertos conocidos)

Ruta: identifica al recurso en el ámbito del anfitrión.

Consulta: Pares ?Clave1=Valor1&Clave2=Valor2… que identifica al recurso de forma paramétrica

Fragmento: Parte de un recurso

Introducción: URI



https://calendar.google.com/calendar/r/week?tab=mc

Esquema: protocolo de acceso (http, https, ftp, ftps, smtp, …)

Host: identificación jerárquica que asigna la autoridad de nombres.

Puerto: número entre 0 y 216-1 (65 535) (puertos conocidos)

Ruta: identifica al recurso en el ámbito del anfitrión.

Consulta: Pares ?Clave1=Valor1&Clave2=Valor2… que identifica al recurso de forma paramétrica

Fragmento: Parte de un recurso

Introducción: Principios REST

Un recurso puede tener varias URI

- http://www.google.com
- https://www.google.com
- http://google.com

Un recurso puede tener varias representaciones

- `XHTML`, `HTML`, `TXT`, `XML`, `JSON`, `JPEG`, …

Un recurso puede tener enlaces a otros recursos (HATEOAS)

- https://en.wikipedia.org/wiki/HATEOAS

Contenido

HTTP: Conceptos básicos

Internet: conjunto de redes de comunicación que permite conectar dispositivos usando el protocolo TCP/IP

WWW: servicio dentro de Internet para poder visualizar páginas Web.

HTML: lenguaje utilizado para describir páginas Web.

HTTP: protocolo de nivel (OSI) de aplicación sobre TCP/IP que permite transferencias de información en la WWW

HTTP: Conceptos básicos

Hypertext Transfer Protocol o HTTP es el protocolo de aplicación estándar usado para la comunicación en la Web.

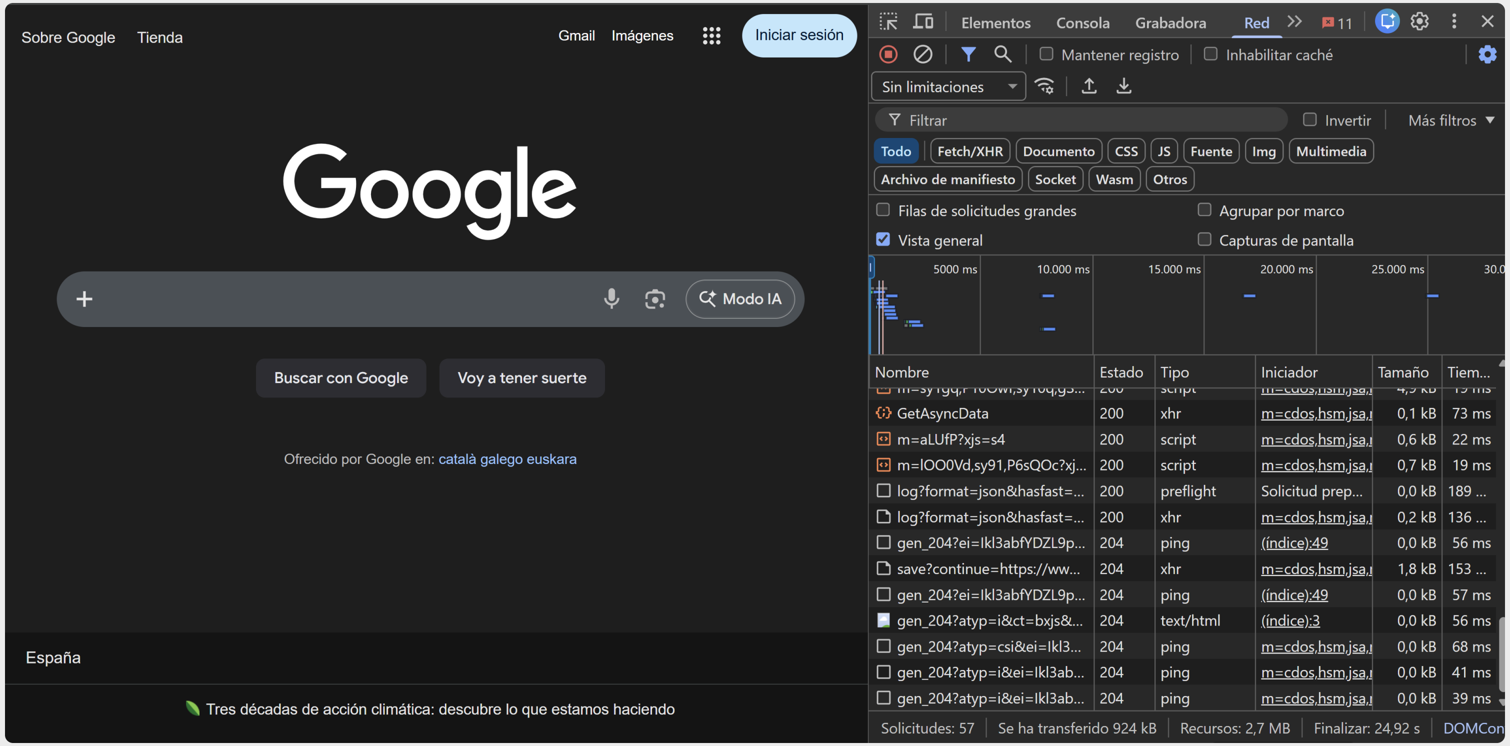
Sigue el esquema petición-respuesta entre un cliente y un servidor.

HTTPS funciona bajo una capa de cifrado pero los mensajes son idénticos

HTTP: El navegador

¿Qué ocurre en el navegador cuando solicitamos una página web (F12)?

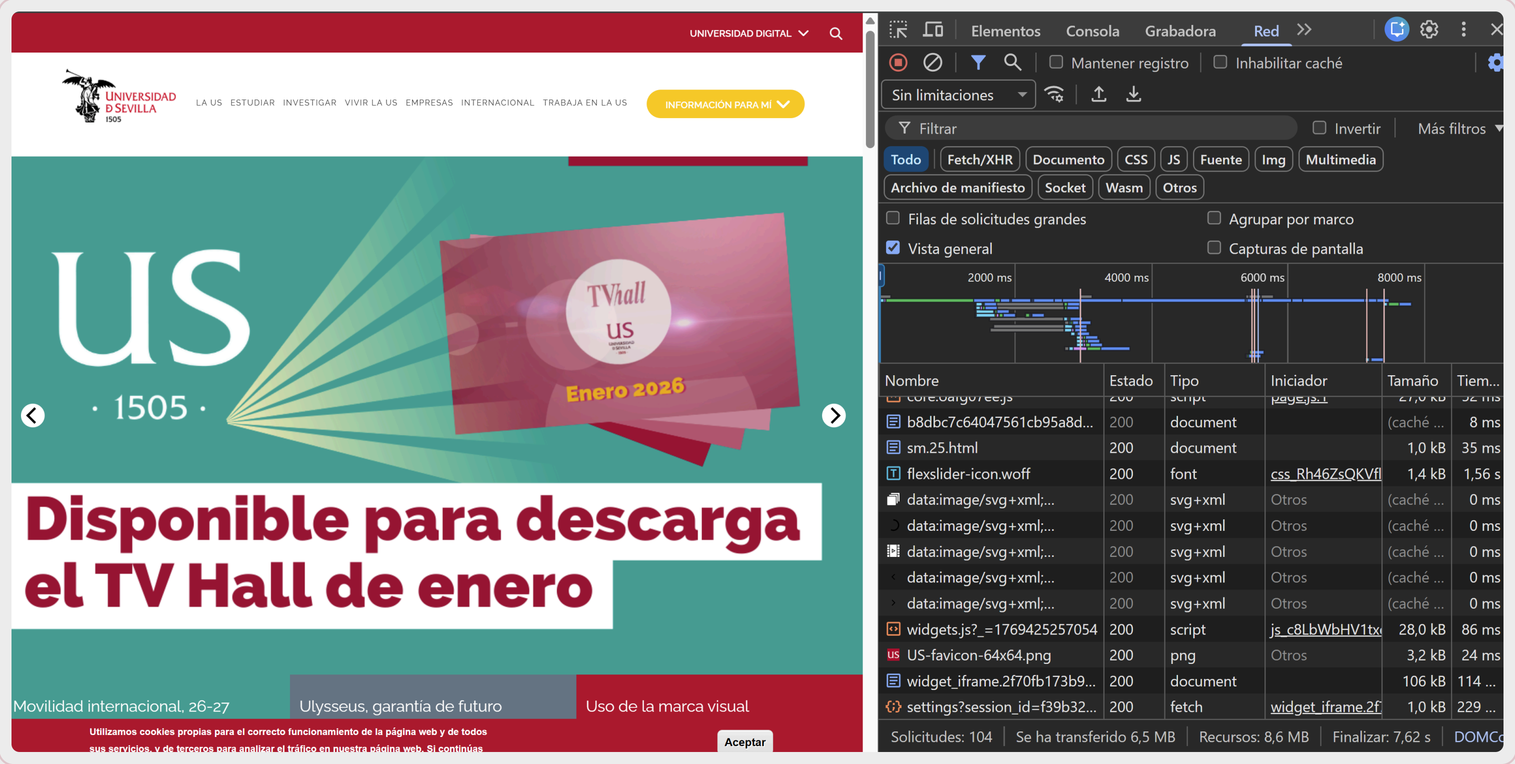
www.google.com -> 57 peticiones HTTP



HTTP: El navegador

¿Qué ocurre en nuestro navegador cuando solicitamos una página web?

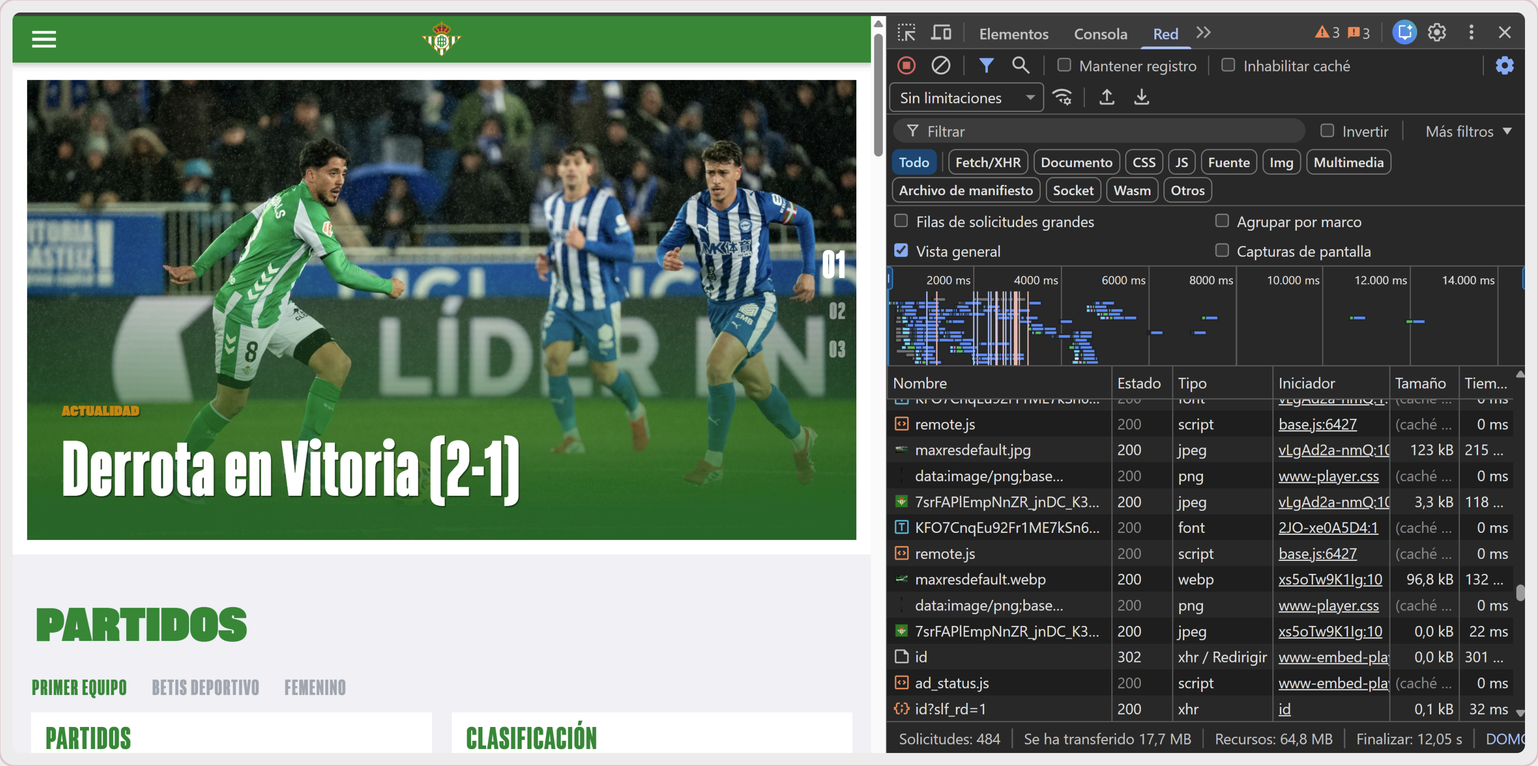
www.us.es -> 104 peticiones HTTP (requests)



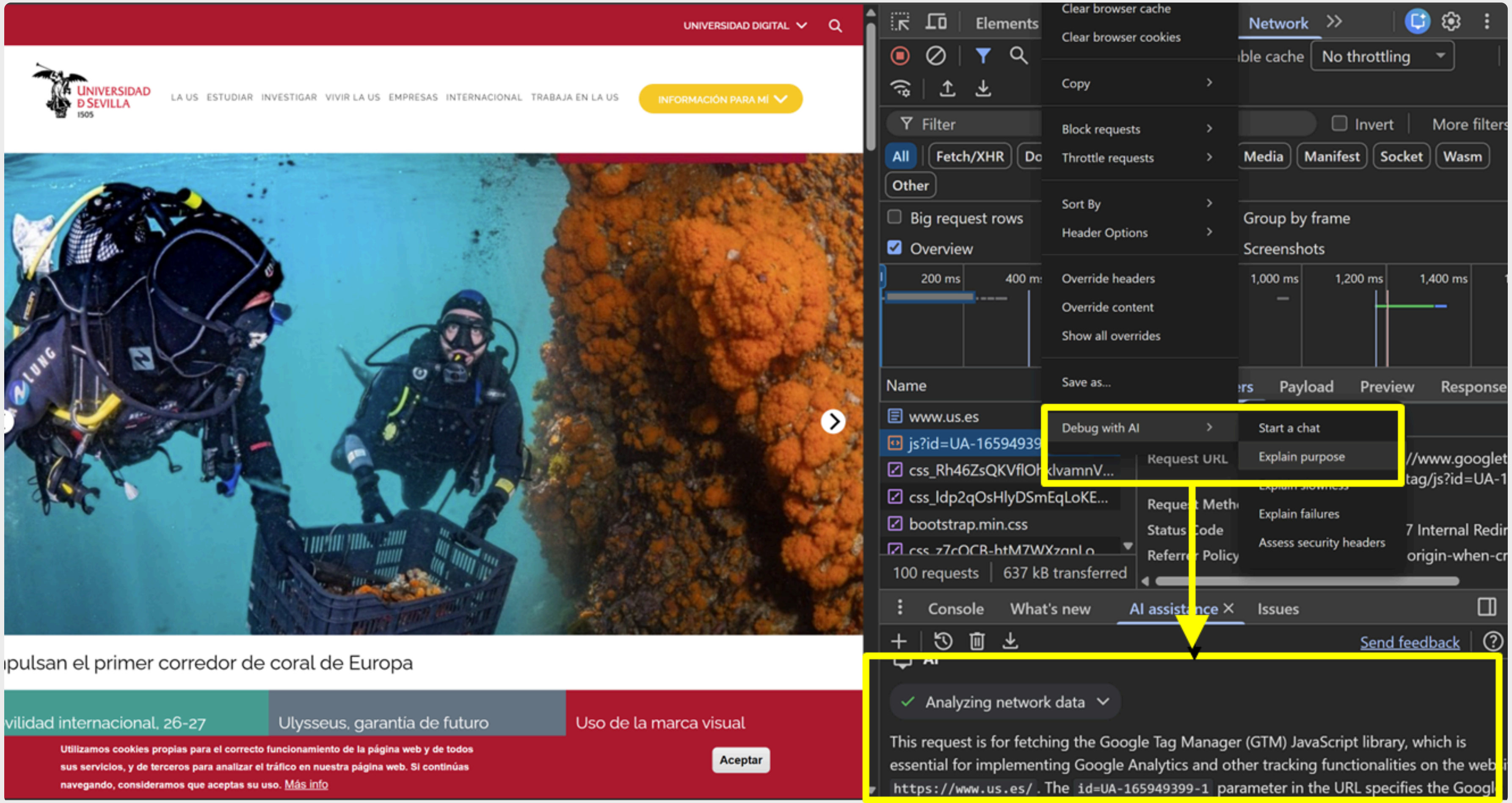
HTTP: El navegador

¿Qué ocurre en nuestro navegador cuando solicitamos una página web?

www.realbetisbalompie.es -> 484 peticiones HTTP (requests)

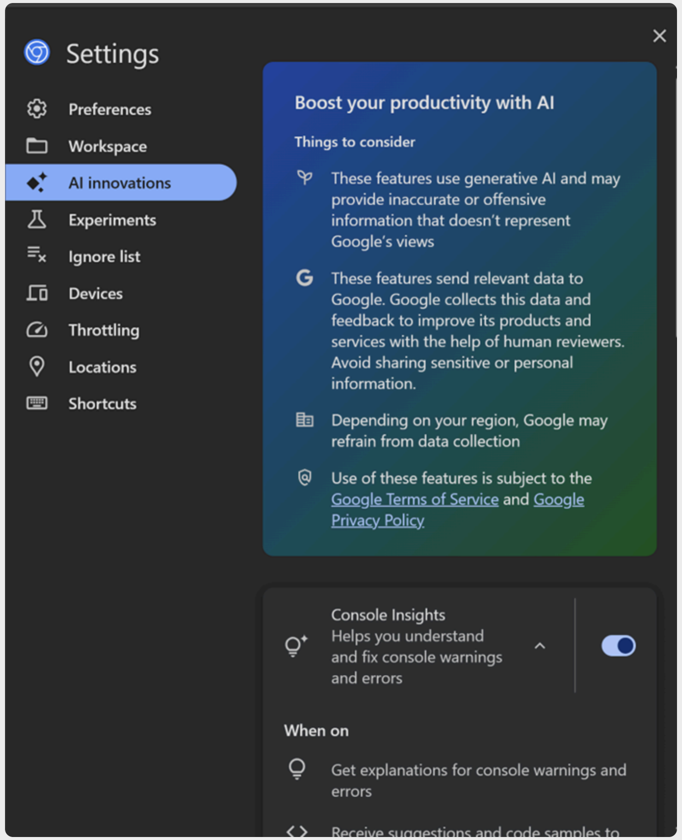


Ayuda con IA (Google Chrome)



Activación (Google Chrome)

Es necesario configurar el idioma de DevTools en inglés



HTTP: Request (header/body)

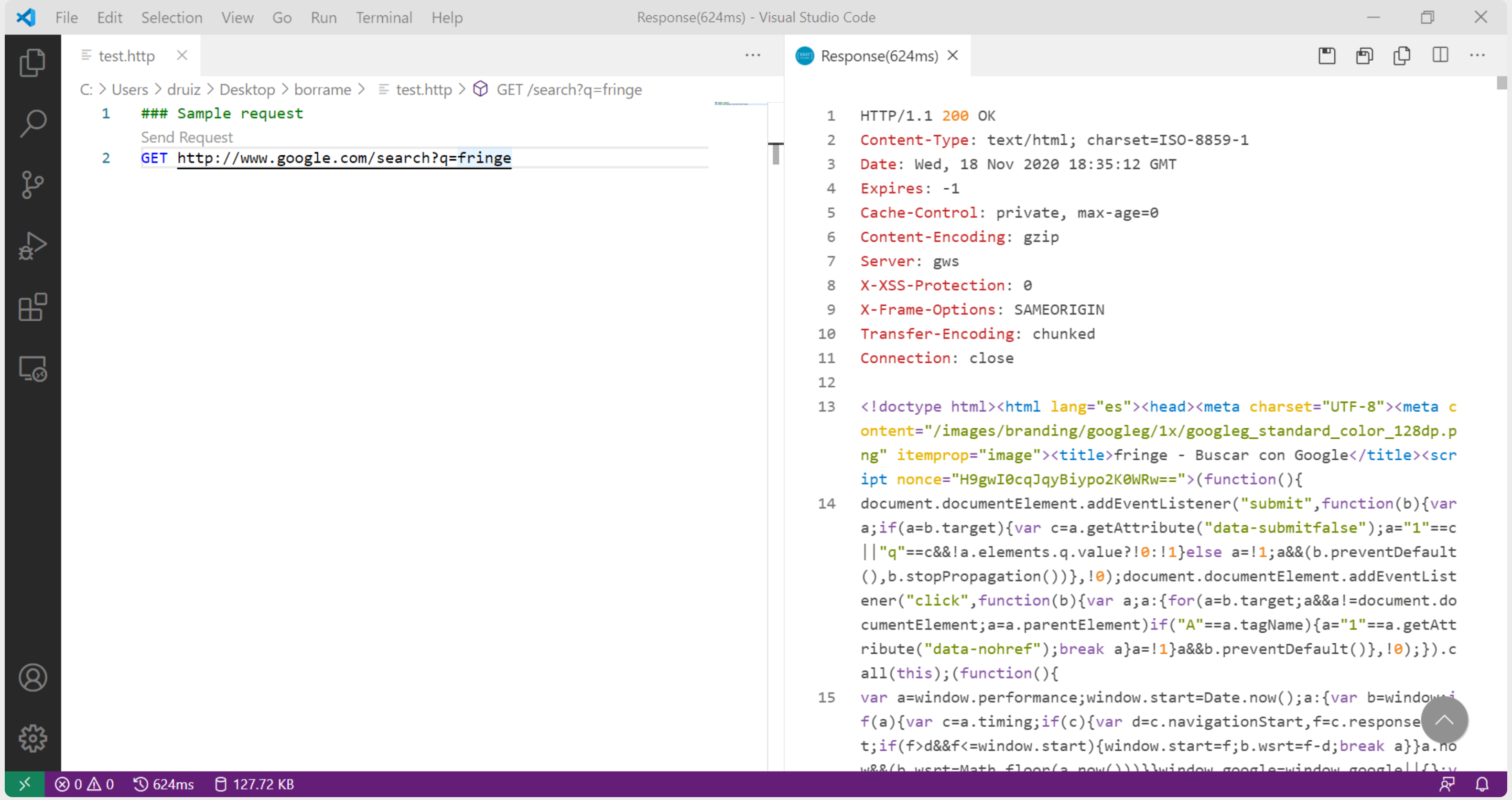
¿Qué formato tiene la petición (http request)?

```
GET  HTTP/1.1
Host: www.google.com
User-Agent: PostmanRuntime/7.19.0
Accept: */*
Cache-Control: no-cache
Postman-Token: 02c2d340-3b50-4b3d-9f8e-df8dbd60f34b,d5a166b7-06c7-48ee-a9e1-f68fb7218742
Accept-Encoding: gzip, deflate
Cookie: 1P_JAR=2019-11-06-20; NID=190=Q0OCx_r_jNAtdYFzMSOvO2mS5hAEGN47ddFMLJJT0vDo_D6EHdYKaKnK5evIwHQfGwi9cM9AjYIAJz-
B3zUCw81RMPdQM_bZHZiU4U6rlljrbLZlp3bIsFtUx25-KqwFUSh1cJ-rm9g6TU_fncOpiiyxsT_UUdKvbz1E2azPdyI
Connection: keep-alive
cache-control: no-cache
```

```
GET  HTTP/1.1
Host: www.us.es
User-Agent: PostmanRuntime/7.19.0
Accept: */*
Cache-Control: no-cache
Postman-Token: 76aa2e60-8fea-4edb-8e25-d4126943a1f8,903aa2ef-9c75-4033-9104-11bcd7265bb2
Accept-Encoding: gzip, deflate
Connection: keep-alive
cache-control: no-cache
```

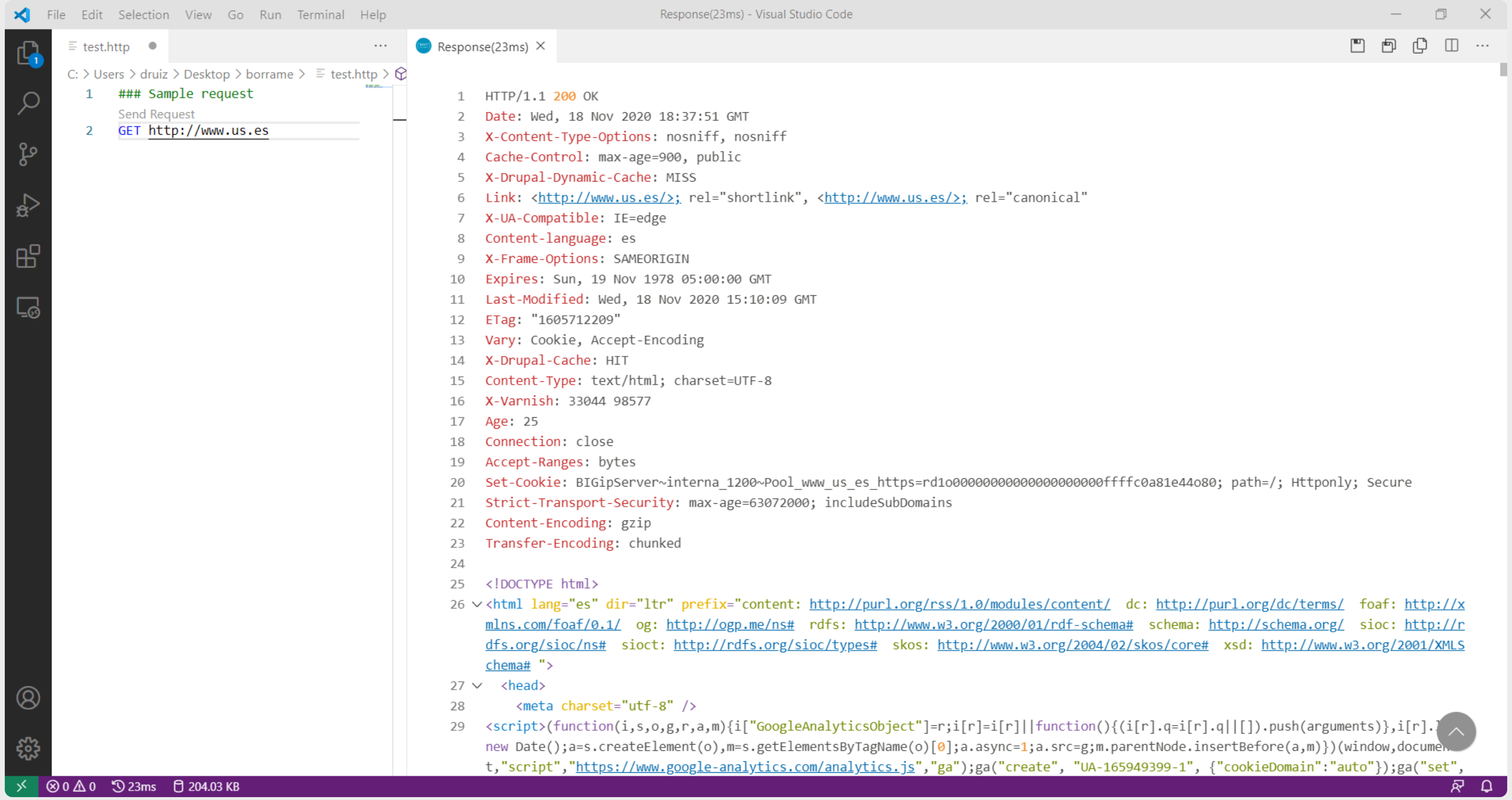
HTTP: Response (header/body)

¿Qué formato tiene la respuesta (http response www.google.com)?



HTTP: Response (header/body)

¿Qué formato tiene la respuesta (http response www.us.es)?



HTTP: Response (header/body)

¿Qué formato tiene la respuesta (http response www.realbetisbalompie.es)?

betis.http

C: > Users > inmah > Downloads > betis.http

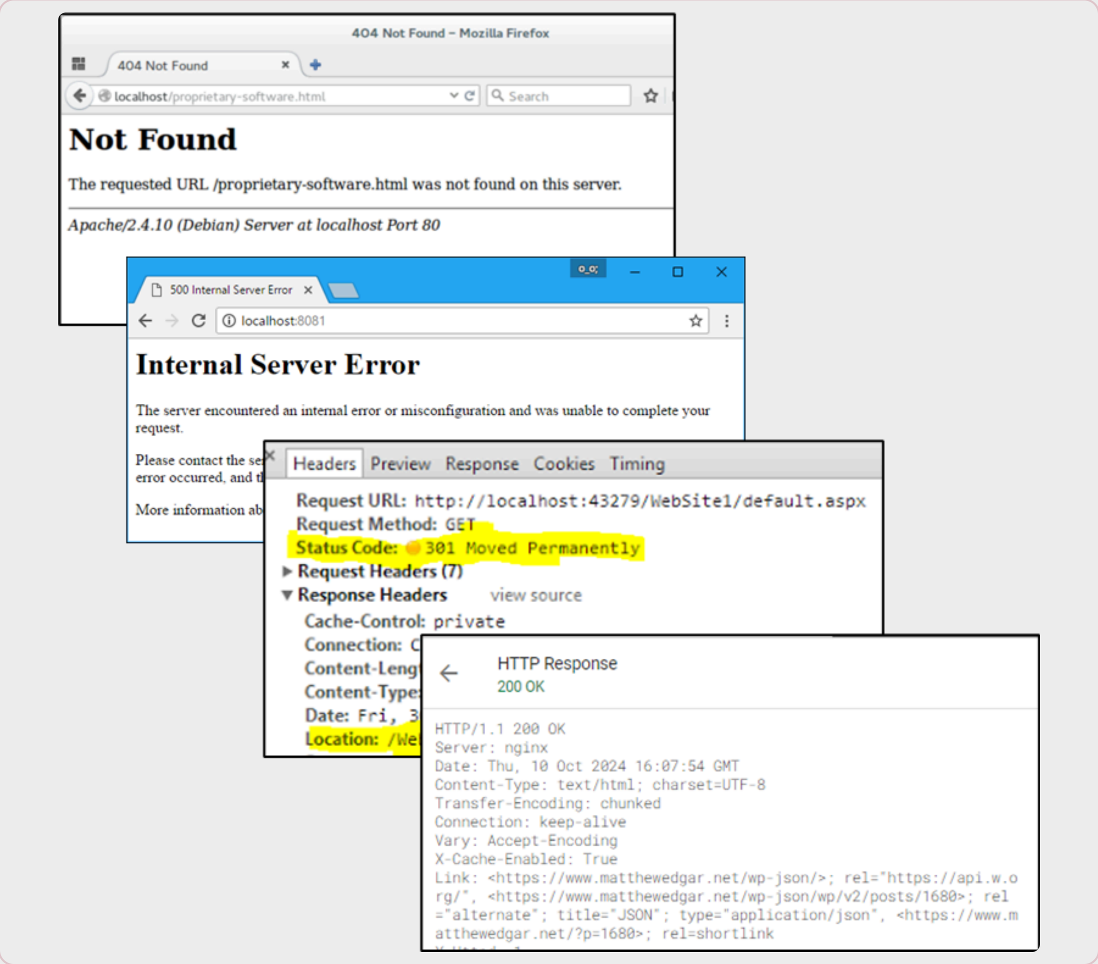
Send Request

```
GET www.realbetisbalompie.es
```

Response(197ms)

```
HTTP/1.1 200 OK
Server: nginx
Date: Mon, 26 Jan 2026 11:24:02 GMT
Content-Type: text/html; charset=UTF-8
Transfer-Encoding: chunked
Connection: close
Vary: Accept-Encoding
Strict-Transport-Security: max-age=31536000; includeSubDomains
X-Frame-Options: SAMEORIGIN
X-Content-Type-Options: nosniff
Permissions-Policy: camera=none, microphone=none, geolocation=none
Set-Cookie: PHPSESSID=jfk38ee1udv7vsc6gs8uselq1l; path=/
Pragma: no-cache
Last-Modified: Mon, 26 Jan 2026 11:24:02 GMT
Expires: Mon, 26 Jan 2026 11:24:32 GMT
Cache-Control: max-age=30
Memcached: YES
Time: 2.4 ms
Memory: 741.88 kb
Content-Encoding: gzip

<meta name="csrf-token" content="596ef54c7c0322b8a81c543a7634200c5ad175304debac3ea1cabd5fba1904b4"><!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml" xmlns:fb="https://www.facebook.com/2008/fbml" xml:lang="es-ES"
        lang="es-ES">
```

HTTP: Códigos de respuesta



Códigos de respuesta HTTP

- 1XX: Informativa
- 2XX: Correcta
- 3XX: Redirección
- 4XX: Error del cliente
- 5XX: Error del servidor

https://es.wikipedia.org/wiki/Anexo:Códigos_de_estado_HTTP

HTTP: Códigos más comunes

https://en.wikipedia.org/wiki/List_of_HTTP_status_codes

Código	Nombre	Descripción
200	OK	Operación exitosa
201	Created	Recurso creado
204	No content	Operación exitosa pero no se devuelve nada
301	Moved permanently	El recurso se ha movido a otra URI permanentemente
307	Moved temporarily	El recurso se ha movido a otra URI temporalmente
400	Bad request	La petición no es válida
401	Unauthorized	El usuario no ha iniciado sesión
403	Forbidden	El usuario no tiene permisos
404	Not found	El recurso solicitado no existe
405	Method not allowed	El método HTTP no está permitido para ese recurso
500	Server error	El servidor ha tenido un error
503	Service unavailable	El servicio solicitado no se encuentra disponible

HTTP: Métodos

GET: Obtiene una representación del recurso especificado. (C**R**UD)

POST: Envía datos al servidor para su procesamiento. Puede resultar en la creación de nuevo recurso. (**C**RUD)

PUT: Actualiza totalmente el recurso especificado. (CR**U**D)

PATCH: Actualiza parcialmente el recurso especificado.(CR**U**D)

DELETE: Borra el recurso especificado. (CRU**D**)

HEAD: Pide una respuesta idéntica a la que correspondería a una petición GET, pero sin el cuerpo de la respuesta. Útil para comprobar si la caché sigue siendo válida mediante la cabecera Last-Modified.

OPTIONS: Devuelve los métodos HTTP que el servidor soporta para una URI.

https://es.wikipedia.org/wiki/Protocolo_de_transferencia_de_hipertexto#Métodos_de_petición

HTTP: Versiones

HTTP/0.9 (1991)

- Sólo implementa el comando GET

HTTP/1.0 (1996)

- GET, HEAD, POST

HTTP/1.1 (1999)

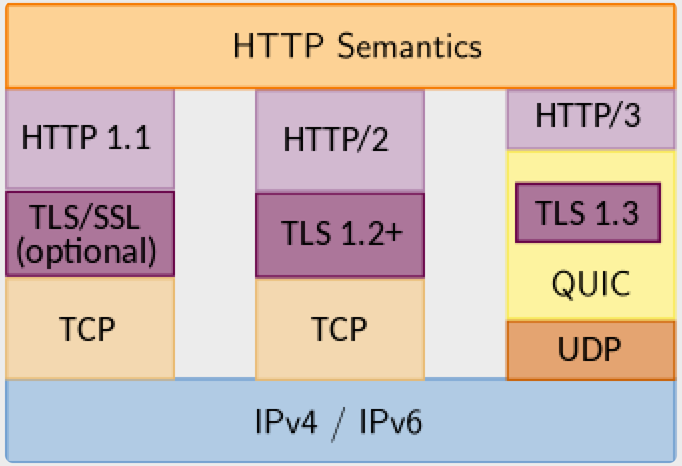
- GET, PUT, POST, DELETE
- Múltiples peticiones a la vez
- Sigue siendo el más usado hoy en día

HTTP/2.0 (2015)

- Mejora el empaquetado y transporte

HTTP/3.0 (2022 – Proposed standard)

- Basado en el protocolo QUIC propuesto por Google
- Usa UDP en lugar de TCP para aligerar las conexiones
- Aún no es estándar (implementado en el 95% de navegadores)



https://en.wikipedia.org/wiki/HTTP/3

Contenido

Introducción

URIs

Protocolo HTTP

Diseño de APIs REST

Consumir APIs REST

Diseño: Conceptos básicos

Los servicios web que implementan los principios de REST se denominan servicios RESTful. De igual forma, las APIs web implementadas a través de servicios RESTful reciben el nombre de APIs RESTful.

Un servicio RESTful permite interaccionar de forma programática con un recurso.

Los servicios RESTful implementan una o varias operaciones CRUD (Create, Retrieve, Update, Delete) sobre el recurso:

Operación	Método HTTP
Crear	POST
Consultar	GET
Actualizar	PUT / PATCH
Eliminar	DELETE

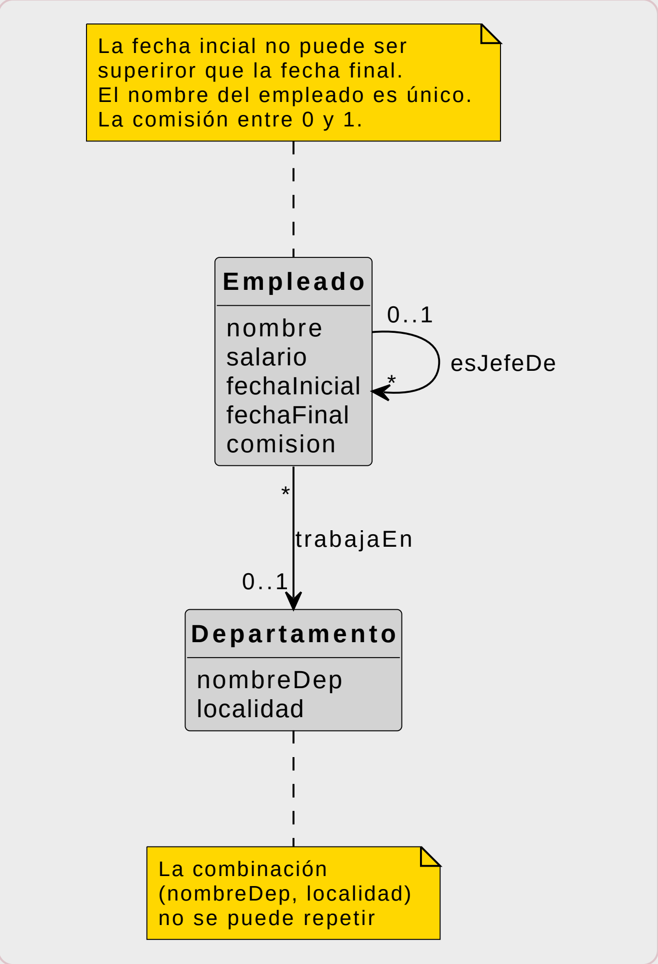
Diseño: Conceptos básicos

Departamentos (base http://ejemplo.es/api/v1)		
	URI	**Descripción**
GET	/departamentos	Obtiene todos los departamentos
GET	/departamentos/{id}	Obtiene el departamento con departamentoId={id}
POST	/departamentos	Inserta un nuevo departamento con los valores indicados en el body de la petición
PUT	/departamentos/{id}	Actualiza el departamento con departamentoId={id} con los valores indicados en el body de la petición
DELETE	/departamentos/{id}	Elimina el departamento con departamentoId={id}

Diseño: Buenas prácticas

- Usar nombres en plural, no verbos
- Usar / para representar relaciones
- Usar versionado
- Usar diferentes métodos HTTP para diferentes acciones sobre el recurso
- Usar códigos de respuesta HTTP según el resultado de la operación
- Usar camelCase (https://en.wikipedia.org/wiki/Camel_case)
- Usar parámetros para filtrar, ordenar y paginar resultados

Diseño: Ejemplo. Trazabilidad



Departamentos

PK (departamentoId)
AK (nombreDep, localidad)

departamentoId	nombreDep	localidad

Empleados

PK (empleadoId)
FK (departamentoId) / Departamentos
FK (jefeId) / Empleados

empleadoId	departamentoId	jefeId	nombre	salario	fechaInicial	fechaFinal	comision

Departamentos (base http://ejemplo.es/api/v1)

	URI	Descripción
GET	/departamentos	Obtiene todos los departamentos
GET	/departamentos/{id}	Obtiene el departamento con departamentoId={id}
POST	/departamentos	Inserta un nuevo departamento con los valores indicados en el body de la petición
PUT	/departamentos/{id}	Actualiza el departamento con departamentoId={id} con los valores indicados en el body de la petición
DELETE	/departamentos/{id}	Elimina el departamento con departamentoId={id}

Diseño: Ejemplo. Departamentos

Departamentos (base http://ejemplo.es/api/v1)		
	URI	**Descripción**
GET	/departamentos	Obtiene todos los departamentos
GET	/departamentos/{id}	Obtiene el departamento con departamentoId={id}
POST	/departamentos	Inserta un nuevo departamento con los valores indicados en el body de la petición
PUT	/departamentos/{id}	Actualiza el departamento con departamentoId={id} con los valores indicados en el body de la petición
DELETE	/departamentos/{id}	Elimina el departamento con departamentoId={id}

Diseño: Ejemplo. Empleados

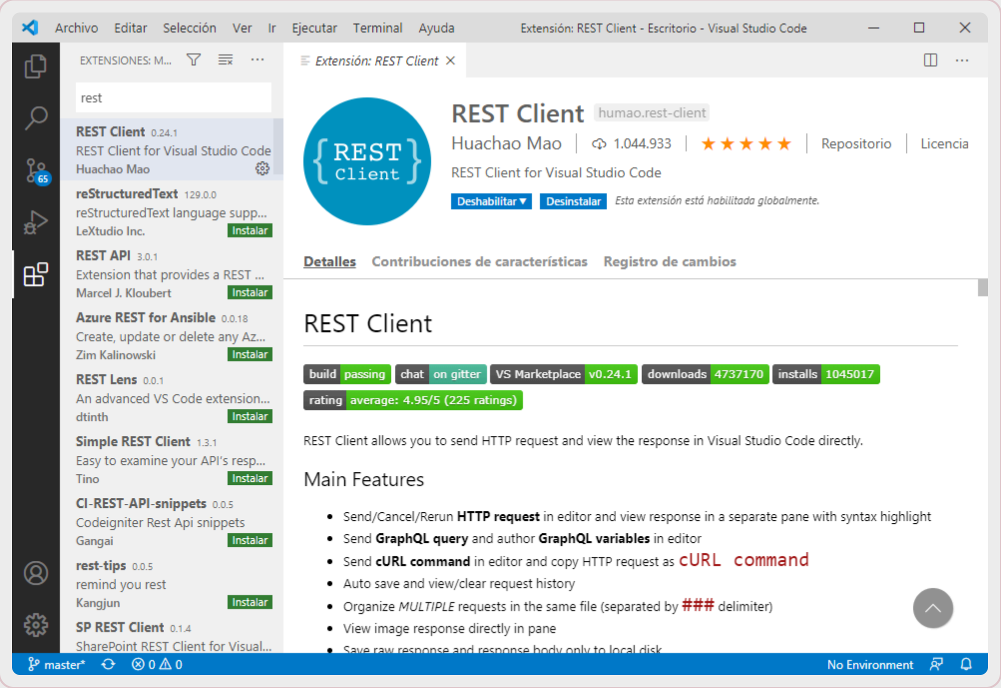
Empleados (base http://ejemplo.es/api/v1)		
Method	**URI**	**Descripción**
GET	/empleados	Obtiene todos los empleados
GET	/empleados?_sort={campo}	Obtiene los empleados ordenados por {campo}
GET	/empleados?_order={asc/desc}	Establece el orden como ascendente o descendente
GET	/empleados?_limit={n1}	Obtiene los n1 primeros empleados
GET	/empleados?_limit={n1}&_page={n2}	Obtiene los n1 empleados a partir de la página n2, con n1 empleados por página.
GET	/empleados/{id}	Obtiene el empleado con empleadoId={id}
GET(*)	/departamentos/{id}/empleados	Obtiene los empleados del departamento con departamentoId={id}
GET(*)	/empleados/{id}/jefe	Obtiene el jefe del empleado cuyo empleadoId={id}
GET(*)	/departamentos/{id}/empleados?{atr1}={c1}&{atr2}={c2}…	Obtiene los empleados del departamento con departamentoId={id} y cuyo valor para el atributo i-ésimo sea {ci}
POST	/empleados	Inserta un nuevo empleado con los valores indicados en el body de la petición
PUT	/empleados/id	Actualiza el empleado con empleadoId={id} cambiando los valores indicados en el body de la petición
DELETE	/empleados/{id}	Elimina el empleado con empleadoId={id}

Contenido

Clientes REST: VS Code

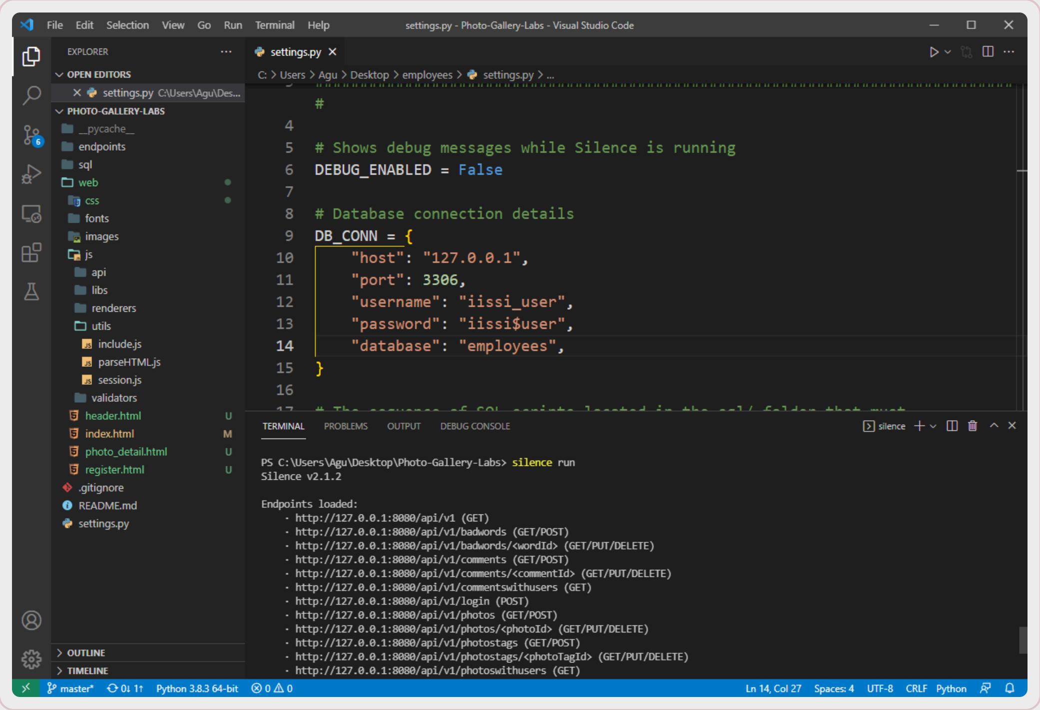
Extensión REST Client para VS Code

Permite hacer peticiones `HTTP` y ver sus respuestas desde el propio editor de código mediante archivos `.http`

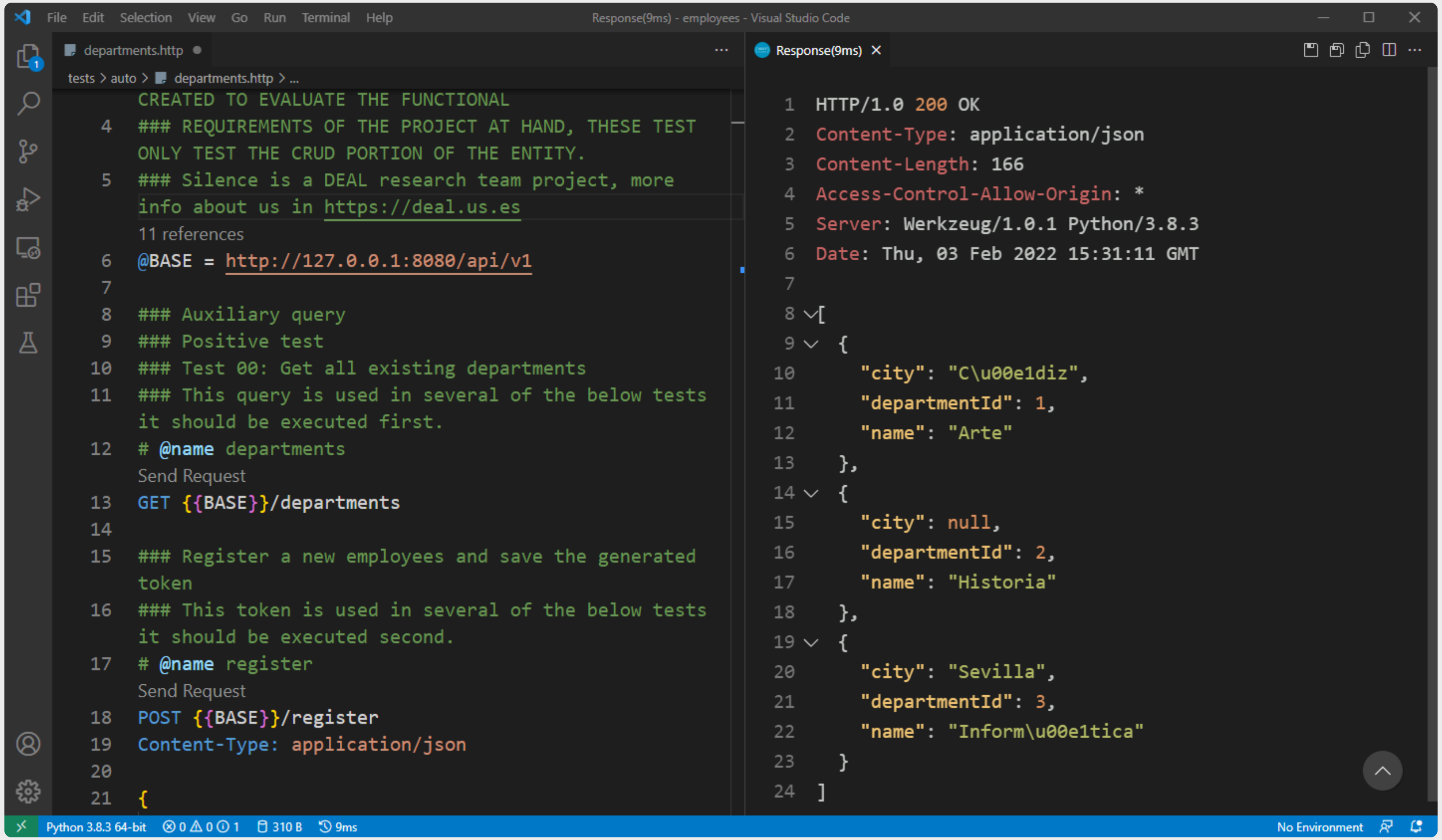


Ejemplo de proyecto Silence

```
# Ejecuta estos comando en el terminal:
> silence new employees --template Employees
> silence createdb
> silence createapi
> silence run
```



Ejemplo: GET


File Edit Selection View Go Run Terminal Help
Response(9ms) - employees - Visual Studio Code
departments.http
tests > auto > departments.http > ...
CREATED TO EVALUATE THE FUNCTIONAL
REQUIREMENTS OF THE PROJECT AT HAND, THESE TEST ONLY TEST THE CRUD PORTION OF THE ENTITY.
Silence is a DEAL research team project, more info about us in https://deal.us.es
11 references
@BASE = http://127.0.0.1:8080/api/v1
Auxiliary query
Positive test
Test 00: Get all existing departments
This query is used in several of the below tests it should be executed first.
@name departments
Send Request
GET {{BASE}}/departments
Register a new employees and save the generated token
This token is used in several of the below tests it should be executed second.
@name register
Send Request
POST {{BASE}}/register
Content-Type: application/json
{
Response(9ms)
HTTP/1.0 200 OK
Content-Type: application/json
Content-Length: 166
Access-Control-Allow-Origin: *
Server: Werkzeug/1.0.1 Python/3.8.3
Date: Thu, 03 Feb 2022 15:31:11 GMT
[
{
"city": "C\u00e1diz",
"departmentId": 1,
"name": "Arte"
},
{
"city": null,
"departmentId": 2,
"name": "Historia"
},
{
"city": "Sevilla",
"departmentId": 3,
"name": "Inform\u00e1tica"
}
]
Python 3.8.3 64-bit
310 B
9ms
No Environment

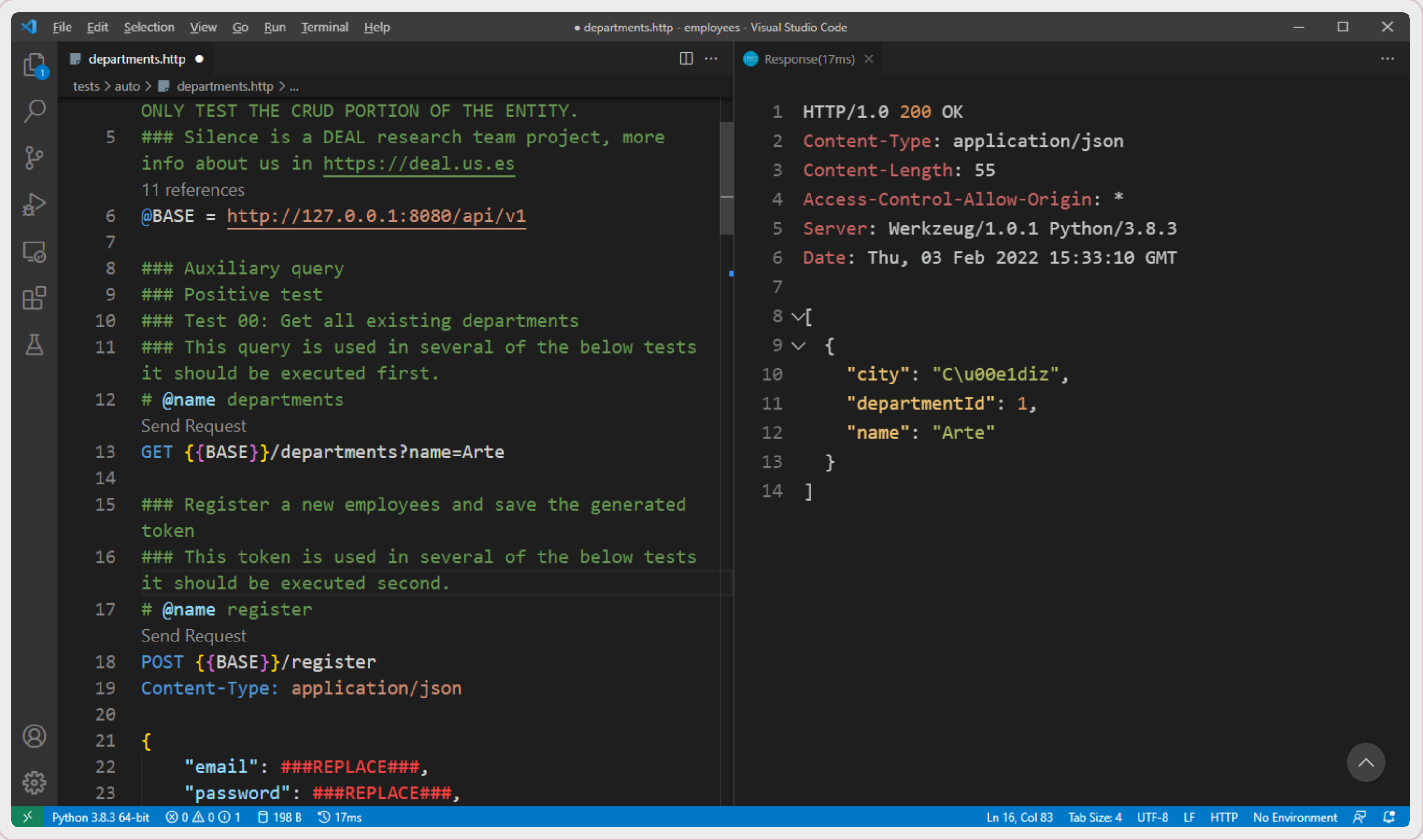
Ejemplo: GET con variable

```
ONLY TEST THE CRUD PORTION OF THE ENTITY.
### Silence is a DEAL research team project, more info about us in https://deal.us.es
@BASE = http://127.0.0.1:8080/api/v1

### Auxiliary query
### Positive test
### Test 00: Get all existing departments
### This query is used in several of the below tests it should be executed first.
# @name departments
GET {{BASE}}/departments?name=Arte

### Register a new employees and save the generated token
### This token is used in several of the below tests it should be executed second.
# @name register
POST {{BASE}}/register
Content-Type: application/json

{
    "email": ###REPLACE###,
    "password": ###REPLACE###,
```

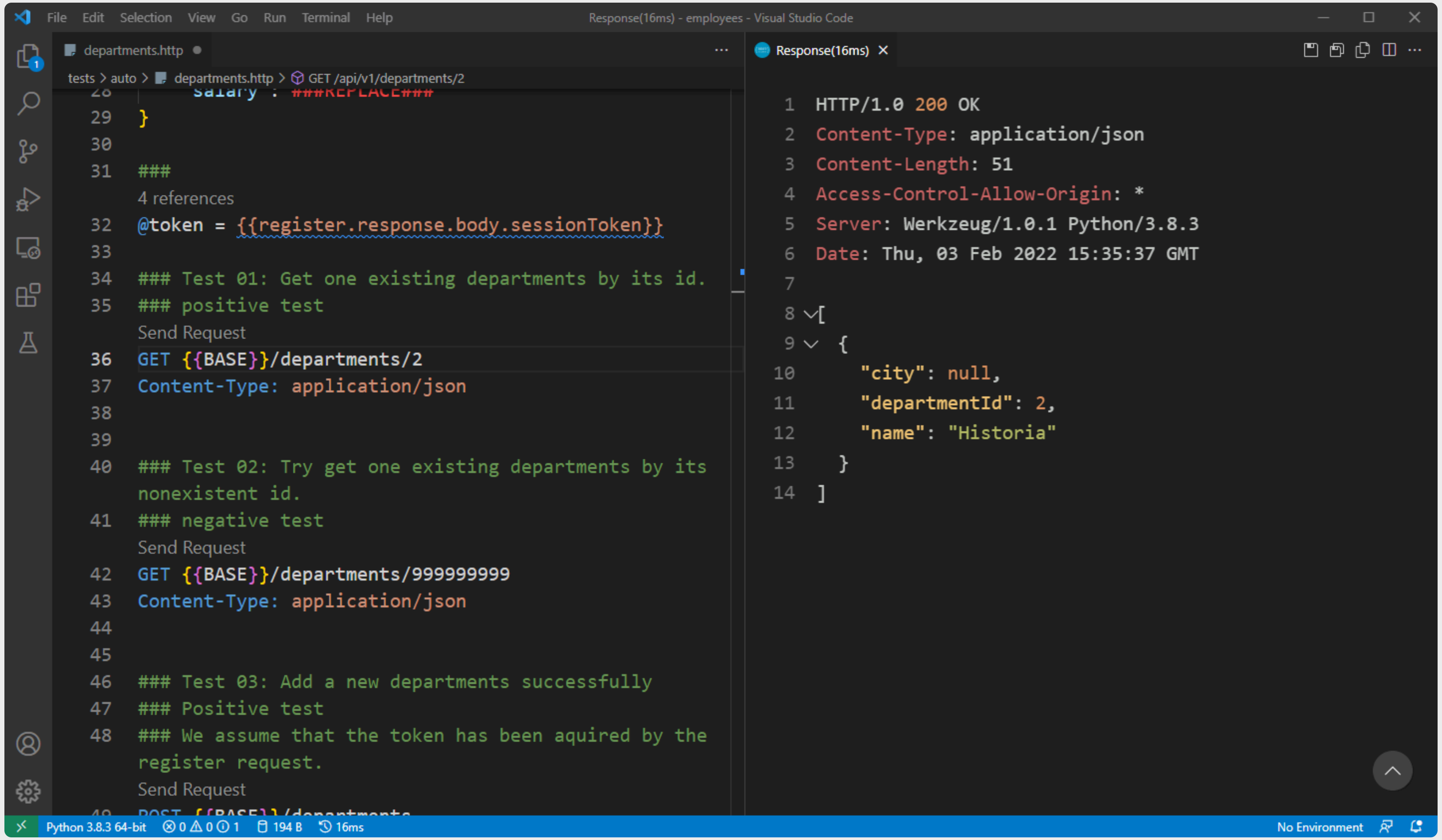
```
HTTP/1.0 200 OK
Content-Type: application/json
Content-Length: 55
Access-Control-Allow-Origin: *
Server: Werkzeug/1.0.1 Python/3.8.3
Date: Thu, 03 Feb 2022 15:33:10 GMT

[
  {
    "city": "C\u00e1diz",
    "departmentId": 1,
    "name": "Arte"
  }
]
```

Ejemplo: GET con parámetros

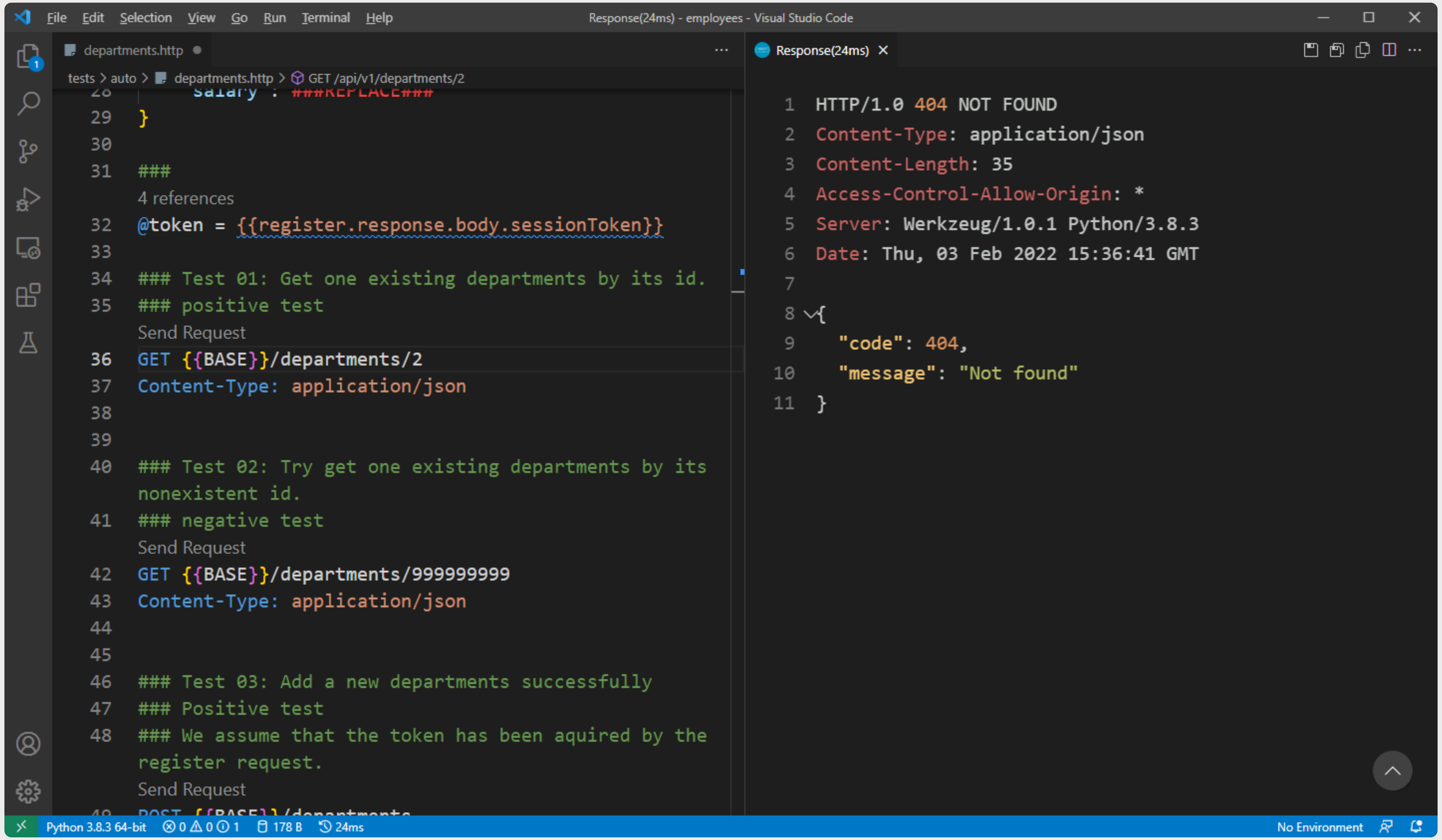
Los parámetros van en la query string



Ejemplo: GET por ID



Ejemplo: GET por ID



Ejemplo: POST – Solicitar token

File Edit Selection View Go Run Terminal Help — Response(103ms) - employees - Visual Studio Code

departments.http — tests > auto > departments.http > login

```
Send Request
GET {{BASE}}/departments?name=Arte

### Register a new employees and save the generated token
### This token is used in several of the below tests it should be executed second.
# @name login
Send Request
POST {{BASE}}/login
Content-Type: application/json

{
    "email": "perianez@company.com",
    "password": "perianez"
}

###
4 references
@token = {{login.response.body.sessionToken}}

### Test 01: Get one existing departments by its id.
### positive test
Send Request
GET {{BASE}}/departments/2
Content-Type: application/json
```

Response(103ms)


```
HTTP/1.0 200 OK
Content-Type: application/json
Content-Length: 488
Access-Control-Allow-Origin: *
Server: Werkzeug/1.0.1 Python/3.8.3
Date: Thu, 03 Feb 2022 15:38:44 GMT

{
  "sessionToken": ".eJxNjbFuxCAMhl_lhG6sTkBCSDJ16dClaqcuXQw2anQQEORU5aq-eyFDVQ-29en_9X0zE0t5Rjaf1pv3DyeGlCBvgdbtoKIiCrD4-rNEeYGV7o82hgTrfqmXHYHk407013BLLtsLBGqtV8IcW8zDf5iXjxvnJNp2gu4tkaCUr5jxkJkrOjmXT5BqmIXidc5PPAzu_a0_a4Xc9kaAFRLkhGaaZKels0Z1goNRo8LOOT2OStheD1oJ5GaUqkdnJVCzFfCQ9-qSHecX_vMLVExQRw.Yfv3BA.EqjcPytGfkzxEYlPCl5ynkAX2qk",
  "user": {
    "bossId": null,
    "departmentId": 1,
    "email": "perianez@company.com",
    "employeeId": 1,
    "firstName": "Pedro",
    "lastName": "Peri\u00e1\u00f1ez",
    "salary": 2300.0
  }
```


Python 3.8.3 64-bit ⊗ 0 ⚠ 0 632 B 103ms No Environment

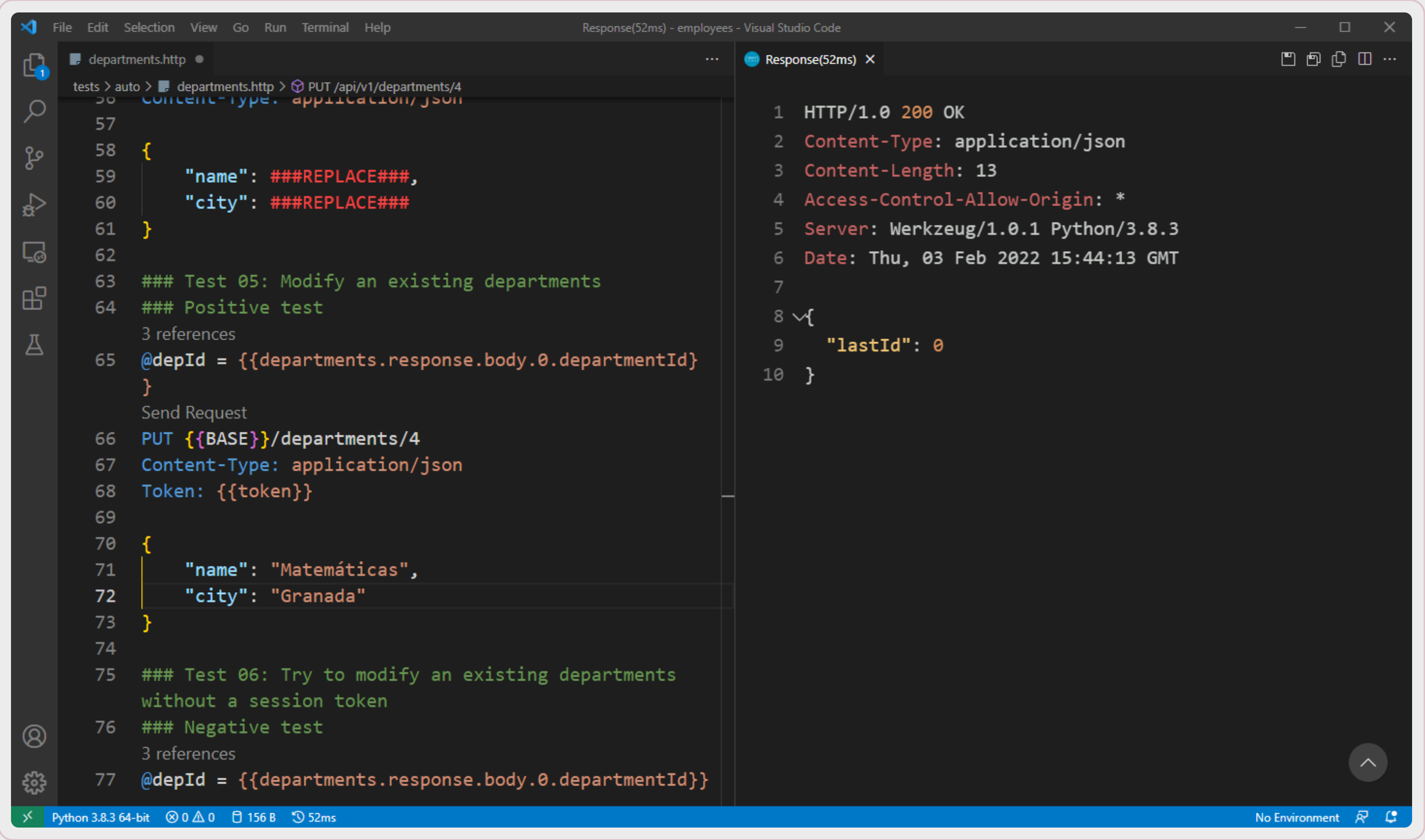
Ejemplo: POST - Insertar

```
### Test 02: Try get one existing departments by its nonexistent id.
### negative test
Send Request
GET {{BASE}}/departments/999999999
Content-Type: application/json


### Test 03: Add a new departments successfully
### Positive test
### We assume that the token has been aquired by the register request.
Send Request
POST {{BASE}}/departments
Content-Type: application/json
Token: {{token}}

{
    "name": "Matemáticas",
    "city": "Málaga"
}

### Test 04: Add a new departments without a session token
### Negative test
Send Request
POST {{BASE}}/departments
```

Response(93ms)

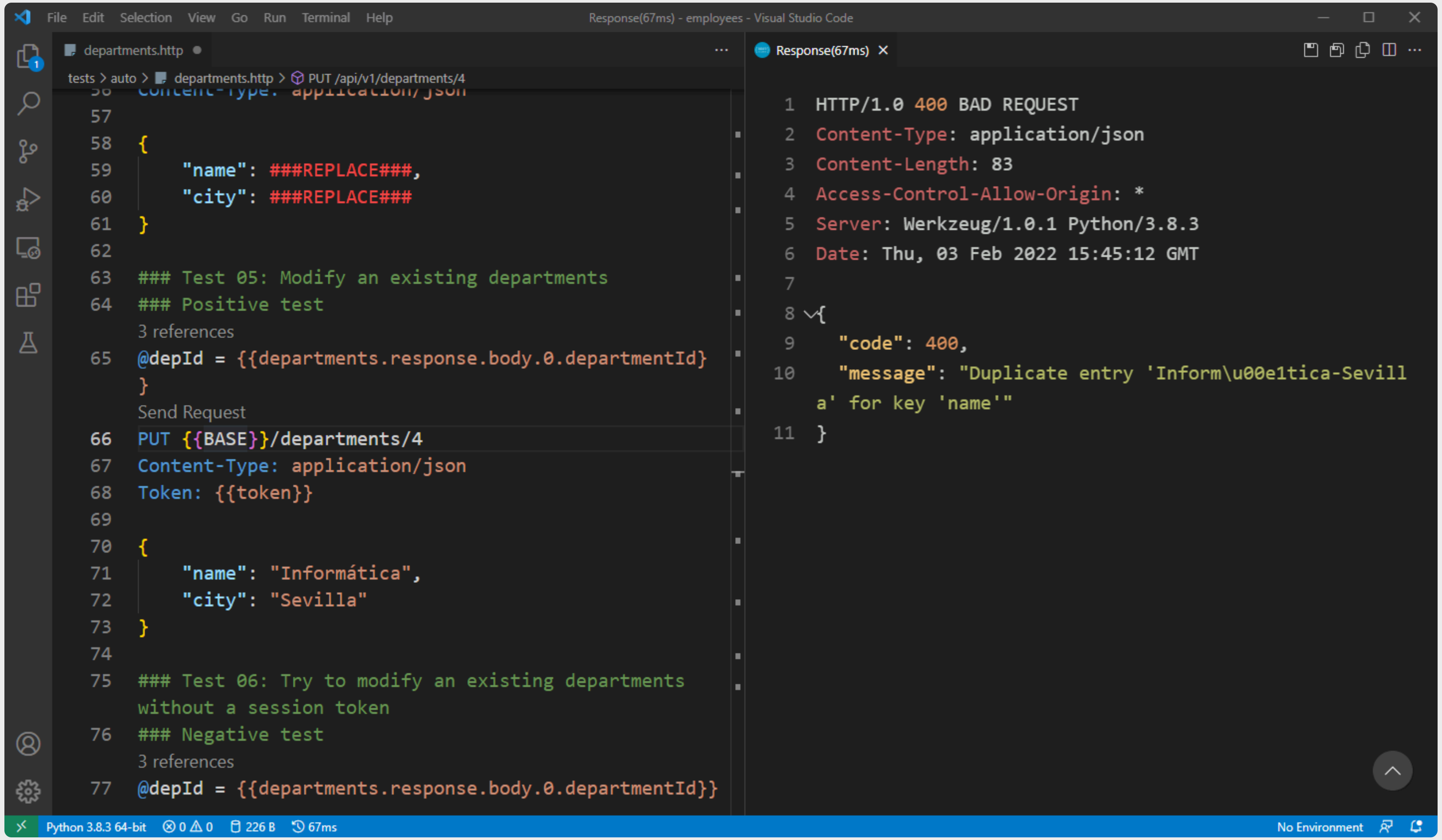
```
HTTP/1.0 200 OK
Content-Type: application/json
Content-Length: 13
Access-Control-Allow-Origin: *
Server: Werkzeug/1.0.1 Python/3.8.3
Date: Thu, 03 Feb 2022 15:43:13 GMT

{
  "lastId": 4
}
```

Ejemplo: PUT - Actualizar

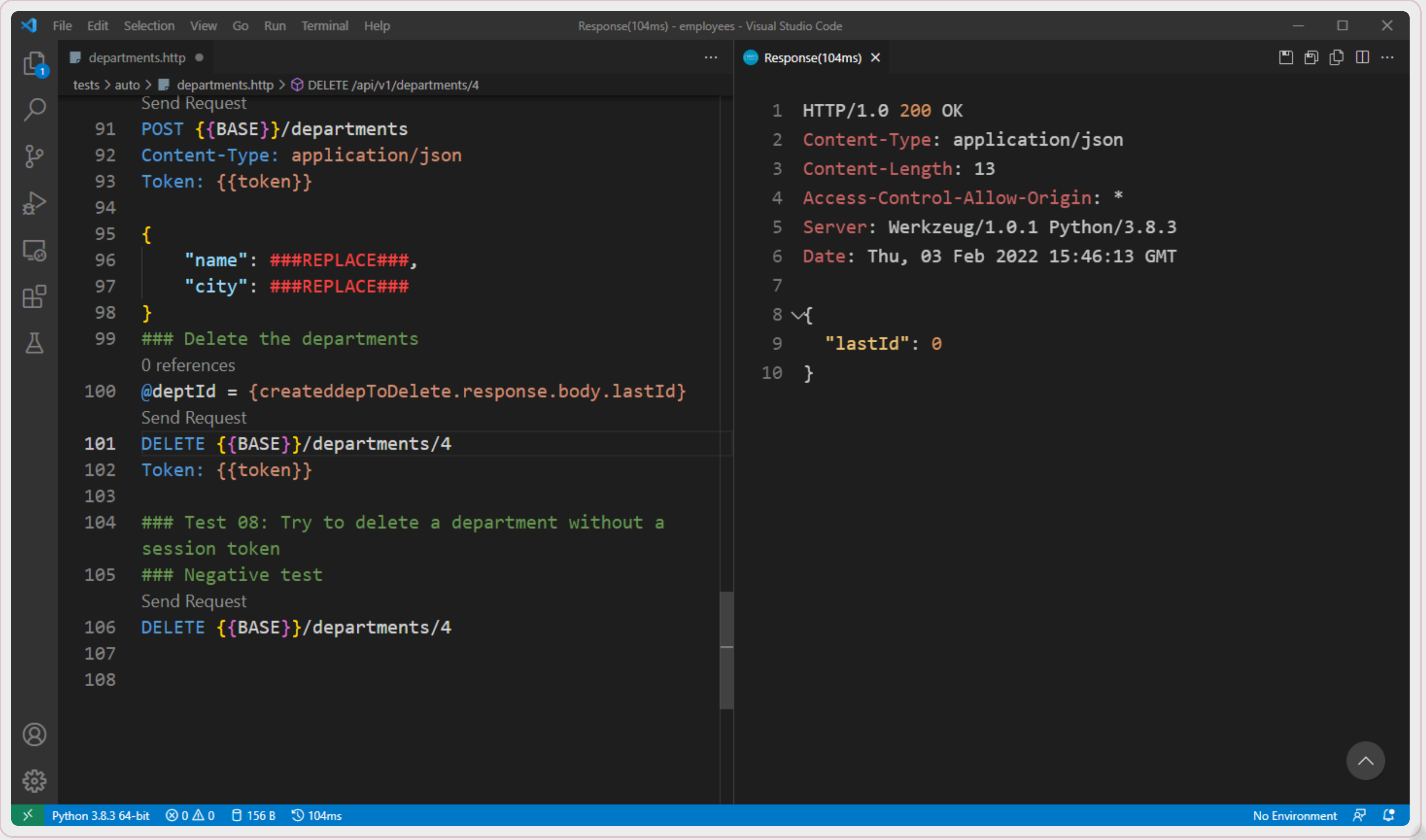


Limitación de MySQL/MariaDB, devuelbe lastId=0 cuando se actualiza

Ejemplo: PUT - Actualizar



Ejemplo: DELETE



Gracias

Universidad de Sevilla

Departamento de Lenguajes y Sistemas Informáticos